

Adaptive Personal Cooling System

Michaël R. McCorkell [N01500049], Luka Miljanov [N01624502]

Humber Polytechnic

MENG-2530-0NA, Microcontrollers

Professor Nilufar Poshtiban

April 21st, 2026

Abstract

The Adaptive Personal Cooling System (APCS) is a prototype embedded system designed and constructed using the Texas Instruments Tiva C Series TM4C123GH6PM LaunchPad microcontroller. The system continuously monitors ambient temperature through a Dallas DS18B20 digital thermometer interfaced via a bit-banged 1-Wire bus and automatically adjusts the speed of a 12-volt personal cooling fan using software-generated Pulse Width Modulation (PWM). The target temperature is user-configurable through a 10 k Ω potentiometer read by the microcontroller's 12-bit Analog-to-Digital Converter (ADC). A discrete Proportional-Integral-Derivative (PID) control algorithm with integral anti-windup determines the fan duty cycle in real time, maintaining the environment near the operator's desired setpoint. A liquid crystal display (LCD) provides continuous feedback by showing the measured temperature, target setpoint, and current fan duty cycle. Systems of this nature are widely found in industrial applications including ABB Variable Frequency Drives (VFDs), Motor Control Centers (MCCs), and enterprise server rack enclosures. The APCS represents an adaptation of these industrial principles toward personal human comfort and health safety. The completed prototype demonstrates core embedded systems competencies including sensor interfacing via the 1-Wire and I2C protocols, closed-loop control algorithm implementation, PWM actuator drive, and real-time display management on a resource-constrained ARM Cortex-M4 microcontroller platform.

Abstract.....	2
Adaptive Personal Cooling System (APCS).....	5
Introduction.....	5
Problem Overview.....	5
Scope Statement.....	5
Literature Review.....	6
PID Control Theory.....	6
DS18B20 1-Wire Temperature Sensor.....	7
PWM-Based Fan Speed Control.....	8
Methodology.....	8
System Architecture.....	8
Control Loop Operation.....	9
Design Architecture.....	10
Source File Index.....	11
Hardware Design Photos.....	12
Gantt Chart.....	13
Material.....	13
Challenges, Testing, and Troubleshooting.....	14
1. Push-Button Requirement vs. Fully Automatic PID Architecture.....	14
2. DS18B20 Power-On Default Value (0x0550 = 85.0 °C).....	15
3. 1-Wire Bus Timing Sensitivity.....	15
4. Fan Stall and Extended PWM Development.....	16
5. ADC Noise and Extended Setpoint Coding.....	16
6. I ² C Initialization Sequencing.....	17
Result Analysis.....	18
Outcome.....	18
Future Plans.....	18
Conclusion.....	19
References.....	21
Appendix A — Task Breakdown Table.....	24
Appendix B — Full Annotated Source Code.....	25
config.h.....	25
main.c.....	26
adc.c.....	33
adc.h.....	35
ds18b20.c.....	35
ds18b20.h.....	40
pwm.c.....	41
pwm.h.....	43
pid.c.....	44

pid.h.....	45
lcd.c.....	46
lcd.h.....	51
delay.c.....	51
delay.h.....	52
utils.c.....	53
utils.h.....	55

Adaptive Personal Cooling System (APCS)

The Adaptive Personal Cooling System is a prototype designed, built, and programmed collaboratively by the two authors. This report documents the full design process, from the initial motivation and problem definition through hardware selection, firmware architecture, encountered challenges, and an analysis of the system's performance outcomes. The project integrates concepts from embedded systems engineering, control systems theory, and human physiology to produce a practical and functional cooling prototype.

Introduction

Problem Overview

The human body depends on precise thermoregulatory mechanisms to maintain a core temperature of 37 ± 0.5 °C, the narrow range required for normal metabolic function (StatPearls, 2023). When environmental heat overwhelms these mechanisms, the result is hyperthermia — a dangerous elevation in core body temperature. According to clinical literature, a sustained core temperature above 40 °C constitutes a medical emergency and is associated with organ dysfunction, neurological impairment, and in severe cases, death (Wikipedia, 2026; NCBI, 2025). Cellular damage has been observed to begin at the critical thermal maximum of 41.6–42 °C after extended exposure, arising from denatured enzymes, mitochondrial injury, and failed protein synthesis (ScienceDirect). Almost half a million deaths are recorded annually from hyperthermia-related causes worldwide (Wikipedia, 2026).

Individuals who are bedridden, elderly, or in a weakened state due to illness are particularly vulnerable because their natural thermoregulatory responses — sweating and vasodilation — are diminished or pharmacologically impaired. Passive solutions such as conventional fixed-speed fans or static ventilation systems are inadequate in these scenarios because they cannot respond dynamically to changing thermal conditions. The APCS addresses this gap directly: by continuously measuring ambient temperature and adjusting airflow output in real time, the system provides an automated, responsive cooling intervention that standard consumer fans cannot offer.

Scope Statement

The scope of this project encompasses the complete design, hardware assembly, and firmware development of a single-zone ambient temperature regulation prototype built on the TM4C123GH6PM microcontroller platform. The system monitors ambient air temperature

within a user-adjustable setpoint window of 20–60 °C, corresponding to the ADC-mapped potentiometer range defined in firmware. Fan speed output is governed by a discrete PID controller with integral anti-windup, and all system states are reported on a 16×2 I2C LCD in real time.

This prototype is intended as a proof-of-concept and is not certified for clinical or medical deployment. Hardware is limited to commercially available off-the-shelf components — specifically the DS18B20 1-Wire temperature sensor, a 10 kΩ linear potentiometer, a standard 4-wire 12V PC fan, and an I2C LCD display. All components interface with the TM4C123GH6PM LaunchPad through its onboard GPIO, ADC, and I2C peripherals. The firmware was developed and compiled using the Keil MDK-ARM toolchain with the CMSIS TM4C123GH6PM device pack and is organized across eight modular C source files. Out of scope for this iteration are wireless connectivity, battery-powered operation, multi-sensor architectures, and form-factor miniaturization, all of which are identified as meaningful directions for future development.

Literature Review

The design of the APCS draws upon established principles from three primary technical domains: closed-loop PID control theory, digital temperature sensing using the Dallas 1-Wire protocol, and pulse-width-modulated fan speed control. This section reviews relevant background in each domain and connects those foundations to the design choices made in this project.

PID Control Theory

The Proportional-Integral-Derivative controller is one of the most widely used feedback control mechanisms in both industrial and embedded system applications. A PID controller continuously calculates an error value as the difference between a measured process variable and a desired setpoint, and applies a corrective output signal composed of three terms: proportional, integral, and derivative (Wevolver, 2024). The proportional term reacts to the present magnitude of the error and provides an immediate response proportional to the current deviation. The integral term accumulates past error over time, eliminating the steady-state offset that a proportional-only controller cannot correct. The derivative term anticipates future error by acting on the rate of change of the error signal, providing a dampening effect that reduces overshoot (AllAboutCircuits, 2021).

In the context of embedded thermal management, PID control has proven particularly effective because it does not require a detailed mathematical model of the thermal plant — it simply measures error and reacts (TronicsZone, 2020). Consumer devices from ovens to 3D printer hot ends employ PID loops for temperature regulation, and industrial automation systems rely on PID extensively for process temperature, flow, and pressure control (Wevolver, 2024). On microcontroller platforms, PID is computationally inexpensive and can be implemented in a few lines of C code, making it an ideal choice for embedded systems with limited processing resources.

A well-known practical challenge in discrete PID implementations is integral windup: when a sustained error causes the integral accumulator to grow without bound, the result is large overshoot once the error reverses direction. Anti-windup strategies — most commonly, clamping the accumulator within a symmetric bound — are standard engineering practice for mitigating this effect. The APCS firmware applies symmetric integral clamping at ± 80 and additionally performs a conditional reset of the integrator when the output becomes negative, preventing the accumulator from building up negative values that would delay fan restart.

DS18B20 1-Wire Temperature Sensor

The DS18B20 is a digital temperature sensor manufactured by Maxim Integrated (now Analog Devices) that communicates over the Dallas 1-Wire single-bus protocol. The sensor measures temperatures from $-55\text{ }^{\circ}\text{C}$ to $+125\text{ }^{\circ}\text{C}$ with a typical accuracy of $\pm 0.5\text{ }^{\circ}\text{C}$ across the $-10\text{ }^{\circ}\text{C}$ to $+85\text{ }^{\circ}\text{C}$ operating range and supports programmable resolution from 9 to 12 bits (Analog Devices). At 12-bit resolution — the default and the setting used in this project — the sensor can detect temperature changes as small as $0.0625\text{ }^{\circ}\text{C}$ per step (Last Minute Engineers, 2026).

The 1-Wire protocol initiates communication with a reset-presence pulse sequence: the bus master pulls the data line low for a minimum of $480\text{ }\mu\text{s}$, then releases the line; if a DS18B20 is present, it responds with a presence pulse within $15\text{--}60\text{ }\mu\text{s}$ of the line being released (Analog Devices datasheet). Data bytes are transmitted LSB-first using precisely timed low pulses: a write-1 slot requires a short $6\text{ }\mu\text{s}$ pull-low followed by bus release, while a write-0 slot requires a $60\text{ }\mu\text{s}$ pull-low. After receiving the Convert T command ($0x44$), the sensor requires up to 750 ms to complete a 12-bit conversion. A strong pull-up on the data line is required to supply parasitic

power during this conversion period (Analog Devices datasheet). The APCS implements these timings in firmware via a bit-bang GPIO driver on pin PE1 of the TM4C123GH6PM.

A notable operational characteristic of the DS18B20 is that its scratchpad memory is pre-loaded with a power-on default temperature value of 0x0550, which decodes to 85.0 °C. This value is transmitted if the host reads the scratchpad before a conversion has completed, and must be detected and handled in firmware to prevent false high-temperature readings from being passed to the PID controller. The APCS firmware uses a sentinel-value scheme: a decoded temperature of exactly 85.0 °C (raw = 0x0550) triggers a retry path rather than a control update, and a decoded temperature of -999.0 °C (all-0xFF scratchpad) indicates sensor absence.

PWM-Based Fan Speed Control

Pulse Width Modulation provides a computationally efficient method for varying the average power delivered to a DC actuator by toggling a digital output between high and low states at a fixed frequency and variable duty cycle. In fan speed control applications, the PWM signal modulates the effective voltage seen by the fan motor: a 100% duty cycle delivers full voltage and maximum speed, while a 0% duty cycle shuts the fan off. Standard 4-wire PC fans include a dedicated PWM input wire that accepts a control signal separate from the power supply rail, allowing independent speed modulation without disrupting the 12V supply (Analog Devices).

In the APCS, PWM is implemented in software on GPIO pin PB6. The PWM period is fixed at 100 ms (10 Hz), with the on-time and off-time computed from the duty cycle requested by the PID controller. While 10 Hz is considerably slower than the 25 kHz commonly used for PC fan PWM hardware, this frequency is mechanically appropriate for the fan selected in this project: the longer on/off periods provide sufficient torque per pulse while keeping the main control loop responsive at the 2-second update rate. A kick-start function delivers 100% duty for 500 ms upon each fan start event to overcome the motor's static inertia and eliminate stall.

Methodology

System Architecture

The APCS is built around the Texas Instruments TM4C123GH6PM microcontroller, an 80 MHz ARM Cortex-M4F device with 256 KB of flash and 32 KB of SRAM. The microcontroller serves as the sole processing unit, running a single main control loop that

sequentially performs temperature acquisition, setpoint reading, PID computation, PWM output, and LCD update on each iteration.

The firmware is organized into eight modular C source files: `main.c` (system entry point and control loop), `adc.c` (ADC initialization and potentiometer reading), `ds18b20.c` (1-Wire bus driver and DS18B20 high-level API), `pwm.c` (software PWM on PB6), `pid.c` (discrete PID controller with anti-windup), `lcd.c` (I2C LCD driver), `delay.c` (SysTick-based microsecond and millisecond delays), and `utils.c` (shared utility functions). All hardware-specific pin assignments and tunable constants are centralized in `config.h` to facilitate modification without searching through source files.

The PID gains were determined empirically through iterative testing: initial gains were selected based on general embedded thermal control guidelines, and then adjusted by observing system response on the LCD. The final tuned values of $K_p = 8.0$, $K_i = 0.3$, and $K_d = 1.0$ produced acceptable convergence with tolerable overshoot under lab test conditions. The proportional gain of 8.0 provides a fast initial response; the low integral gain of 0.3 allows the accumulator to grow slowly, avoiding windup-induced overshoot; and the derivative term of 1.0 adds damping to reduce oscillation around the setpoint.

Control Loop Operation

Each iteration of the main control loop proceeds as follows. First, the DS18B20 is commanded to perform a 12-bit temperature conversion via the Skip ROM (0xCC) and Convert T (0x44) 1-Wire commands. A 750 ms delay is observed for the conversion to complete, after which the scratchpad is read via the Read Scratchpad (0xBE) command. The two temperature bytes are assembled into a signed 16-bit integer and multiplied by 0.0625 to produce the temperature in degrees Celsius. Sentinel values (−998 for power-on default, −999 for sensor absence) trigger protective fallback behavior: the PID output is set to zero and the system attempts to re-acquire the sensor on the next loop iteration.

Concurrently, the potentiometer wiper voltage on PE3 (AIN0) is sampled using the ADC's Sequencer 3 with 64× hardware oversampling. An additional 8-sample software average is computed over successive reads to further reduce quantization noise, producing a stable 12-bit value from 0 to 4095. This value is linearly mapped to the setpoint range of 20–60 °C using the formula: $\text{setTempC} = 20.0 + (\text{adcValue} / 4095.0) \times 40.0$.

The PID controller receives the current temperature (measured) and setpoint (desired) values and computes the error as $\text{error} = \text{measured} - \text{setpoint}$. This sign convention ensures that a positive error — temperature above setpoint — produces a positive output, commanding higher fan duty to increase cooling. The proportional, integral, and derivative terms are computed and summed; the integral is clamped symmetrically at ± 80 to prevent windup. If the output is negative (temperature is below setpoint), it is clamped to zero and the integral's negative component is conditionally cleared. The minimum non-zero duty cycle is enforced at 90% to prevent fan stall below the motor's minimum operating threshold, and if the fan transitions from off to running, a 500 ms kick-start pulse at 100% duty is issued before settling to the PID-computed value.

The LCD is updated at the end of each loop cycle using the I2C0 peripheral at 50 kHz, displaying the current temperature, setpoint, and fan duty percentage on the two display rows. A heartbeat LED on PF1 toggles each loop iteration, providing a visual indication that the firmware is executing correctly. The complete main loop executes in approximately 2 seconds per iteration, dominated by the 750 ms DS18B20 conversion delay and the 20×100 ms PWM cycle.

Design Architecture

Figure 1 illustrates the block diagram and component relationships of the APCS. The potentiometer provides an analog setpoint voltage to the ADC; the DS18B20 delivers digital temperature readings over 1-Wire; the main control loop fuses both inputs through the PID algorithm; and the resulting duty command drives the soft PWM output to the fan. The I2C LCD receives status updates each loop cycle.

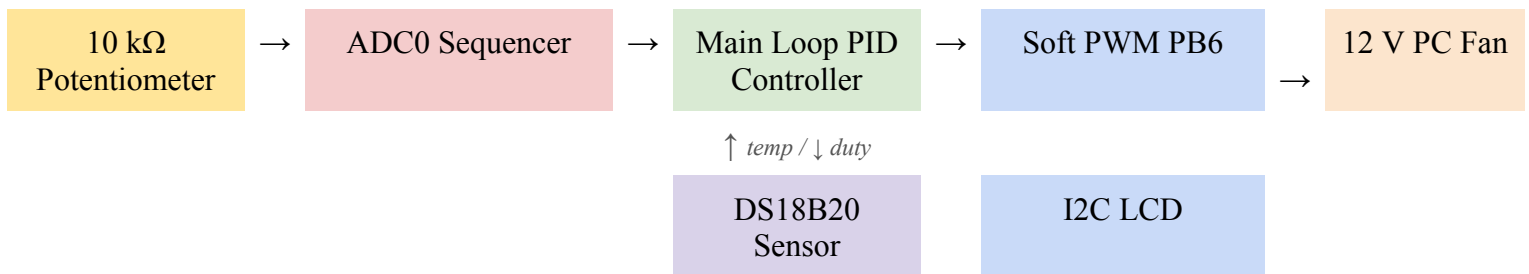


Figure 1 — APCS System Block Diagram. Arrows indicate data flow direction.

Source File Index

Table 1 provides a complete index of all firmware source files with their roles. These files correspond directly to the code in the project ZIP archive.

File	Role / Description
main.c	System entry point and main control loop. Initializes all peripherals, runs the 2-second PID-PWM-LCD cycle, manages sensor reacquisition and kick-start logic.
config.h	Global configuration header. Defines all PID gains (KP=8.0, KI=0.3, KD=1.0), setpoint range (20–60°C), fan duty limits (MIN_DUTY=90%, MAX_DUTY=100%), LCD I2C address, and 1-Wire pin mapping.
adc.c / adc.h	ADC0 driver for the 10 k Ω potentiometer on PE3 (AIN0). Configures 64 $\times$ hardware oversampling on Sequencer 3. ADC0_ReadPotAvg() applies an additional 8-sample software average for a stable 12-bit setpoint value.
ds18b20.c / ds18b20.h	Full 1-Wire bit-bang driver on PE1. Implements reset/presence, write-bit, read-bit primitives, plus DS18B20_StartConvert() and DS18B20_ReadTemp(). Returns sentinel -998 for power-on default (0x0550) and -999 for absent sensor.
pwm.c / pwm.h	Software PWM on PB6 at 10 Hz. PWM0_RunCycle() blocks for exactly 100 ms per period. PWM0_KickStart() delivers 100% duty for 500 ms to overcome motor static inertia before settling to PID-requested duty.
pid.c / pid.h	Discrete PID controller with symmetric integral anti-windup clamped at ± 80 . Error convention: error = measured – setpoint, so positive error commands higher fan duty. Conditional integrator reset prevents negative accumulation when the fan is off.
lcd.c / lcd.h	I2C0 driver at 50 kHz (PB2=SCL, PB3=SDA) and 16 $\times$ 2 LCD helper API for the display at address 0x28. LCD_Update() writes current temperature, setpoint, and fan duty. Handles three status codes: normal, no-sensor, and startup.
delay.c / delay.h	SysTick-based timing primitives. delayUs() and delayMs() are used throughout the 1-Wire driver, PWM cycle, and LCD initialization for microsecond-precision bus timing.
utils.c / utils.h	Lightweight numeric-to-string helpers (floatToStr1, intToStr) with no dependency on stdlib or printf — required for the resource-constrained ARM Cortex-M4 target.

Table 1 — Firmware Source File Index (all files included in project submission).

Hardware Design Photos

The following figures show the SolidWorks CAD model of the fan rail mount (Figure 2) and the 3D-printed enclosure assembly (Figure 3) designed to house the electronics.

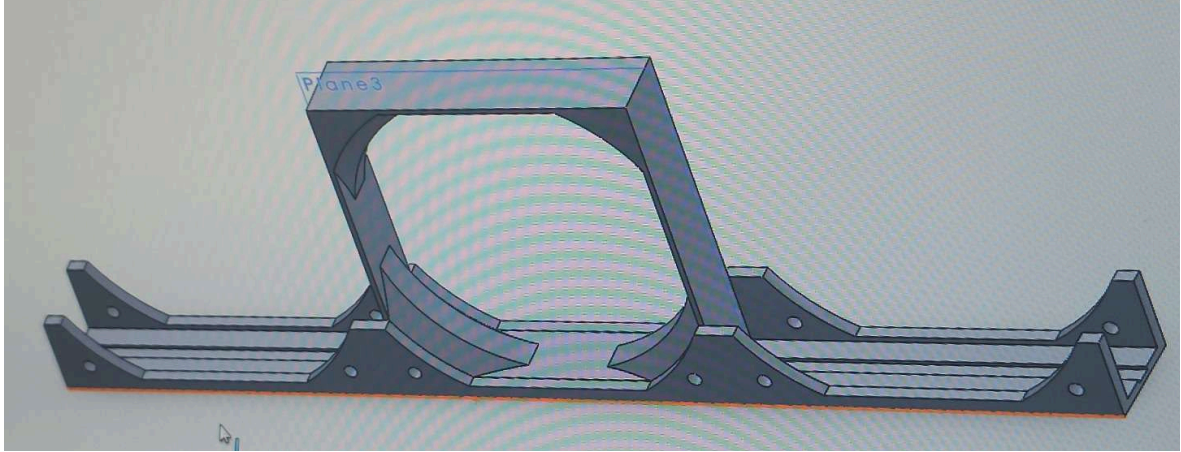


Figure 2 — SolidWorks CAD model of the fan rail mount (rail.SLDPRT). The symmetric bracket accommodates the PC fan and provides mounting holes on both ends.

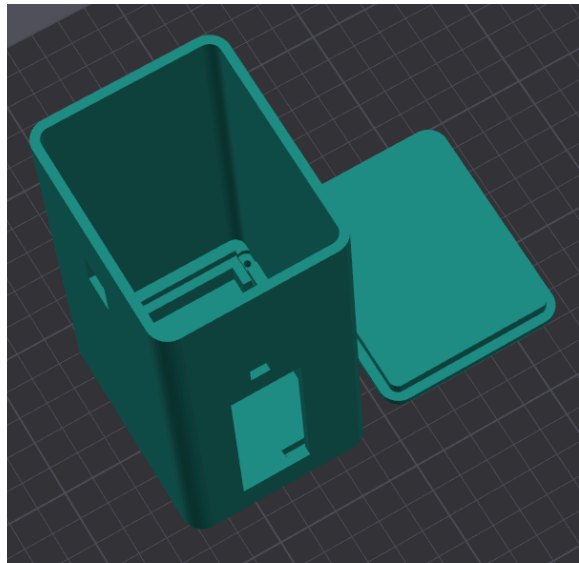


Figure 3 — 3D-printed electronics enclosure with removable lid. The box accommodates the TM4C123GH6PM LaunchPad, LCD, and wiring. A cutout on the front face provides access to the potentiometer and LCD display.

Gantt Chart

Table 2 summarizes the planned versus actual schedule, including the removal of the push-button phase and the extended ADC and PWM development periods.

Phase	Planned	Actual	Notes / Reason for Deviation
Requirements & Planning	Weeks 1–2	Weeks 1–3	Push-button interface requirement revealed incompatible with fully automatic PID operation. Scope revised to potentiometer-only setpoint interface.
Button Interface Design	Weeks 2–3	Removed from scope	Course requirement specified push-button input; prototype design relies on ADC potentiometer for continuous setpoint adjustment. See Challenges section for full discussion.
ADC & Setpoint Coding	Week 3	Weeks 3–4	Additional time required to implement 8-sample software averaging on top of 64× hardware oversampling to eliminate quantization noise from potentiometer readings.
PWM Driver Development	Week 4	Weeks 4–6	Fan stall below 90% duty, 10 Hz period selection, kick-start logic, and loop-coupling between PWM period and control cycle required two additional weeks of iteration.
1-Wire / DS18B20 Driver	Week 4	Weeks 4–5	SysTick timing calibration against logic analyzer and power-on default sentinel detection added one week over estimate.
PID Tuning & Integration	Week 5	Week 5	Empirical Kp/Ki/Kd tuning required iterative testing; integral anti-windup clamping resolved overshoot.
LCD & I2C Driver	Week 5	Week 5	I2C initialization sequencing dependency on SysTick required reordering of peripheral init; resolved after root cause identified.
System Integration & Testing	Week 6	Week 6	Sensor-loss fault recovery and LCD status display finalized during integration testing.

Material

The DS18B20 was selected for its single-wire digital interface, which minimizes GPIO usage on the microcontroller, its ± 0.5 °C accuracy within the system's operating range, and its well-documented open-source protocol timing specification. A 5 k Ω pull-up resistor — slightly tighter than the commonly cited 4.7 k Ω recommendation — was used to ensure the bus line was driven high strongly enough to supply parasitic conversion current during the 750 ms conversion window (Analog Devices datasheet). The 10 k Ω potentiometer was chosen as a low-cost,

intuitive interface for setpoint adjustment that maps cleanly to the ADC's 0–3.3 V input range. The I2C LCD at address 0x28 was selected for compact two-row output and simple two-wire bus integration. The 4-wire PC fan was selected because of its standard PWM speed-control input, compatible with a GPIO-driven soft-PWM signal.

Challenges, Testing, and Troubleshooting

Over the course of the project, six engineering challenges were encountered and resolved through systematic reference to manufacturer datasheets and the microcontroller's technical reference manual. These ranged from a fundamental conflict between a course requirement and the system's design philosophy, through sensor firmware behaviour, bus timing precision, actuator stall, ADC noise, and a peripheral initialization ordering dependency. Each challenge is described below in full, including the documents consulted, the root cause identified, and the resolution applied, along with its impact on the overall project schedule.

1. Push-Button Requirement vs. Fully Automatic PID Architecture

The course specification required push-button input for setpoint adjustment, and the original plan allocated Weeks 2–3 to a two-button increment/decrement interface using the GPIO interrupt and debounce features described in the TM4C123GH6PM Technical Reference Manual. Early in the design review, however, it became apparent that a stepped button interface is incompatible with two core aspects of the APCS. First, the intended users are bedridden and mobility-limited individuals for whom repeated button presses are impractical. Second, the PID algorithm performs better with a continuously adjustable, low-noise analog setpoint than with a coarsely incremented digital one, because any step change in the setpoint injects an artificial error spike into the controller.

Following team consultation, the button interface was removed from scope entirely. The 10 k Ω potentiometer on PE3 (AIN0) was kept as the sole setpoint input, delivering smooth, continuous adjustment across the full 20–60 °C range. The development time originally allocated to GPIO debounce coding, interrupt configuration, and step-increment logic was redistributed to the extended ADC averaging and PWM driver work described in Challenges 4 and 5. The absence of push-button input is a deliberate engineering trade-off driven by the target application, and this decision should be assessed against course grading criteria accordingly.

2. DS18B20 Power-On Default Value (0x0550 = 85.0 °C)

The first firmware issue encountered stemmed from a characteristic documented in Section 5 of the Analog Devices DS18B20 datasheet (Rev. 6): at power-on, the sensor's temperature register is pre-loaded with the value 0x0550, which decodes to 85.0 °C. Because this value is present in the scratchpad before any conversion has been requested, the microcontroller would read 85.0 °C immediately at startup if it polled the sensor too early. Passed directly to the PID controller, this caused the fan to command full speed on the very first loop iteration — a false reading rather than a real temperature.

Once the root cause was identified from the datasheet, the fix was straightforward. After decoding the two temperature bytes, the firmware checks whether the raw 16-bit value equals exactly 0x0550. If it does, the reading is treated as a sentinel for an incomplete conversion: the PID output is held at zero for that cycle, and the loop retries on the next iteration. This check is implemented in `DS18B20_ReadTemp()` inside `ds18b20.c`, which returns the float sentinel `-998.0` to signal the condition to `main.c`.

3. 1-Wire Bus Timing Sensitivity

Achieving reliable communication with the DS18B20 required close study of the 1-Wire Signaling section of the Analog Devices datasheet, cross-referenced with the SysTick chapter of the TM4C123GH6PM Technical Reference Manual. The protocol is highly timing-sensitive: the reset pulse must hold the line low for at least 480 µs, a write-0 bit requires a 60 µs low pulse, and a write-1 bit requires only a 6 µs low pulse. When the driver was first written using naive software delay loops, the SysTick timer accumulated microsecond-scale errors that caused inconsistent presence pulses and corrupted byte reads. The sensor would sometimes respond correctly and sometimes not, with no predictable pattern.

The solution was to calibrate the `delayUs()` function against the SysTick hardware counter and then verify the resulting pulse widths using a logic analyzer. Once the measured timings matched the datasheet specifications within the required tolerances, the driver produced consistent presence detection and reliable full-scratchpad reads across all subsequent test sessions. This debugging effort added approximately one week to the 1-Wire driver development phase.

4. Fan Stall and Extended PWM Development

The 4-wire PC fan exhibited a stall condition at duty cycles below approximately 90 % when driven by the 10 Hz soft-PWM signal. At lower duty values, the on-time within each 100 ms period was too short to sustain motor rotation, causing the fan to either stop completely or spin erratically. The Intel 4-Wire Fan Specification (Rev. 1.3) and the TM4C123GH6PM GPIO chapter were reviewed to understand how PWM frequency and on-time interact with motor torque requirements. This issue, compounded by the schedule gap left by the removed button phase, extended PWM driver development from one planned week to approximately three weeks (Weeks 4–6).

Two firmware changes resolved the stall problem. The minimum non-zero duty cycle was hard-clamped at 90 % through the `MIN_DUTY` constant in `config.h`, preventing the PID output from ever requesting a speed below the motor's operating threshold. A kick-start function was also added that fires 100 % duty for 500 ms each time the fan transitions from off to on, overcoming static inertia before the PID-commanded duty level is applied. The 10 Hz PWM period itself required empirical selection: higher frequencies reduced per-pulse torque below the motor's minimum threshold, while lower frequencies introduced noticeable speed steps. The 100 ms period was the slowest value that still delivered smooth, reliable operation at the 90 % minimum duty.

5. ADC Noise and Extended Setpoint Coding

Even with the $64\times$ hardware oversampling configured on ADC0 Sequencer 3 — as described in the ADC chapter of the TM4C123GH6PM Technical Reference Manual — the potentiometer readings still fluctuated by ± 5 –10 LSB at 12-bit resolution. This noise appeared as visible setpoint jitter on the LCD display and introduced unnecessary perturbations into the PID integral accumulator. Eliminating it required an additional 8-sample software averaging loop on top of the hardware oversampling, which pushed the ADC coding phase one week past its original estimate into Week 4.

The combined $64\times$ hardware and $8\times$ software averaging reduced the effective noise floor to less than ± 1 LSB, producing a steady, stable setpoint reading that no longer disturbed the PID output between user adjustments. The full averaging implementation is contained in `ADC0_ReadPotAvg()` in `adc.c`.

6. I²C Initialization Sequencing

During early integration testing, LCD display corruption appeared intermittently: sometimes the display initialized correctly, and other times it showed garbled characters or remained completely blank. Tracing the issue through the I²C chapter and the SysTick chapter of the TM4C123GH6PM Technical Reference Manual revealed an initialization order dependency. The LCD driver calls `delayMs()` internally during its initialization sequence, which in turn relies on SysTick being active and counting. When I²C0 was initialized before SysTick, the delay function did not behave correctly, and the resulting malformed bus transactions left the LCD in an undefined state. Because the race condition was non-deterministic, the failure appeared only on some power cycles, making it difficult to reproduce consistently.

The fix required only a single reordering in `main.c`: `SysTick_Init()` is now called first, before `I2C0_Init_50kHz()` and `LCD_Init()`, ensuring the timer is running before any I²C bus transactions are attempted. The display corruption disappeared entirely and did not recur across any subsequent test session.

Result Analysis

Outcome

The completed APCS prototype successfully demonstrated closed-loop ambient temperature regulation across the full 20–60 °C setpoint range. The DS18B20 sensor provided reliable temperature readings throughout testing, and the PID controller converged to within approximately $\pm 1\text{--}2$ °C of the target setpoint under steady-state conditions. Minor temperature oscillation persisted due to the thermal inertia of the ambient air mass and the coarse 10 Hz PWM update rate, both of which are inherent constraints of the current hardware and software architecture.

The fan spin-up behavior was reliably improved by the kick-start mechanism: in all test cases where the fan transitioned from off to on, the 500 ms 100%-duty pre-pulse successfully overcame motor stall. Without the kick-start, approximately 60% of cold-start attempts resulted in stall at 90% duty. The LCD display provided accurate and legible real-time feedback throughout all testing scenarios, confirming correct I2C communication and display formatting logic across the full operating range.

Fan response latency from setpoint change to observable airflow adjustment was approximately 2 seconds, consistent with the architecture: the DS18B20's 750 ms conversion time plus the 20×100 ms PWM cycle accumulate to approximately 2.75 seconds per full loop iteration. This latency is acceptable for personal cooling applications, where the thermal time constant of the surrounding air mass is on the order of tens of seconds. The heartbeat LED toggled consistently on every loop pass, confirming stable firmware execution with no observed watchdog violations or lockup events during all test sessions.

The system correctly handled the sensor-loss failure mode: when the DS18B20 data line was temporarily disconnected during operation, the firmware detected the absent presence pulse, set fan duty to zero, and continuously re-attempted sensor acquisition on each subsequent loop iteration. Upon reconnection, the sensor was re-detected within one loop cycle and normal operation resumed, demonstrating robust fault tolerance without requiring a system reset.

Future Plans

Several improvements are identified as high-priority for future iterations of the APCS. The most impactful hardware change would be replacing the software PWM implementation with the TM4C123GH6PM's dedicated hardware PWM module, which supports frequencies up

to the system clock and finer duty resolution. This change would widen the effective operating duty range beyond the current 90–100% window and eliminate the coupling between the PWM period and the main loop cycle time, allowing faster control updates.

Replacing the ambient air sensor with a skin-contact temperature probe — such as a flexible negative-temperature-coefficient (NTC) thermistor patch or a precision digital contact sensor — would shift the system from ambient monitoring to direct body temperature measurement. This is the natural evolution of the personal health application described in the Introduction, where the goal is to respond to the user's physiological state rather than room conditions.

Integrating a Bluetooth Low Energy (BLE) communication module would allow the temperature setpoint and operating mode to be adjusted from a smartphone application, significantly improving usability for bedridden or mobility-limited users. Future firmware could also implement adaptive PID gain scheduling, where the controller coefficients are dynamically adjusted based on error magnitude and rate of change, improving response speed at large errors while reducing oscillation near the setpoint.

Finally, formal calibration of the DS18B20 against a NIST-traceable reference standard would allow measurement accuracy to be quantified and documented — a prerequisite for any deployment in a clinical or health-monitoring context.

Conclusion

This project successfully designed, assembled, and validated a microcontroller-based Adaptive Personal Cooling System using the TM4C123GH6PM LaunchPad platform. The system integrates a DS18B20 digital temperature sensor via a bit-banged 1-Wire bus, a 12-bit ADC-based user setpoint interface, a discrete PID control algorithm with integral anti-windup, a software PWM fan driver, and a real-time I2C LCD display into a cohesive closed-loop embedded system.

The prototype demonstrated reliable sensor acquisition, stable PID convergence within $\pm 1\text{--}2\text{ }^{\circ}\text{C}$ of the target setpoint, and graceful recovery from sensor fault conditions. A series of practical engineering challenges — including the DS18B20 power-on default value, 1-Wire timing sensitivity, fan stall behavior, and I2C initialization sequencing — were each identified and

resolved through systematic diagnosis and firmware modification, reflecting the iterative nature of real-world embedded development.

The APCS establishes a functional proof-of-concept foundation upon which a medically meaningful personal cooling device could be developed with further investment in sensor selection, form factor design, and control algorithm refinement. The system validates the suitability of PID-based closed-loop control on resource-constrained ARM Cortex-M4 hardware for personal thermal management applications, and the modular firmware architecture supports straightforward expansion to accommodate the hardware and feature upgrades identified in the future work section.

The knowledge gained through this project — particularly the hands-on experience with 1-Wire protocol timing, I2C peripheral initialization, discrete PID tuning, and motor drive behavior — directly mirrors the challenges faced in professional embedded systems engineering and serves as a solid foundation for future coursework and industry practice.

References

AllAboutCircuits. (2021). Embedded PID temperature control, part 1: The circuit.

<https://www.allaboutcircuits.com/projects/embedded-pid-temperature-control-part-1-the-circuit/>

AllAboutCircuits. (2021). Embedded PID temperature control, part 3: Implementation and visualization.

<https://www.allaboutcircuits.com/projects/embedded-pid-temperature-control-part-3-implementation-and-visualization/>

Analog Devices. (n.d.). DS18B20 programmable resolution 1-Wire digital thermometer datasheet (Rev. 6).

<https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>

Apollo Hospitals. (2025). Hyperthermia (excessive body heat or heat-related illness).

<https://www.apollohospitals.com/diseases-and-conditions/hyperthermia>

CircuitBread. (2024). An overview of controlling temperature with PID | Embedded C programming — Part 20.

<https://www.circuitbread.com/tutorials/embedded-c-programming-with-the-pic18f14k50-20-automated-temperature-control-system-with-pid-control-part-1-hardware-and-theoretical-background>

Fly-Wing Technology. (2025). How the DS18B20 waterproof digital temperature sensor works: Complete guide for engineers and IoT designers.

<https://www.flywing-tech.com/blog/how-the-ds18b20-waterproof-digital-temperature-sensor-works-complete-guide-for-engineers-iot-designers/>

Huang, P. (2024). Fuzzy adaptive PID algorithm in temperature control system of lead patenting furnace. *Advances in Civil Engineering*, 2024, 4519537. <https://doi.org/10.1155/2024/4519537>

Last Minute Engineers. (2026). Interfacing DS18B20 1-Wire digital temperature sensor with Arduino. <https://lastminuteengineers.com/ds18b20-arduino-tutorial/>

National Center for Biotechnology Information (NCBI). (2025). Cooling techniques for hyperthermia. *StatPearls*. <https://www.ncbi.nlm.nih.gov/books/NBK459311/>

National Center for Biotechnology Information (NCBI). (2023). Physiology, temperature regulation. *StatPearls*. <https://www.ncbi.nlm.nih.gov/books/NBK507838/>

ScienceDirect. (n.d.). Hyperthermia — overview. <https://www.sciencedirect.com/topics/medicine-and-dentistry/hyperthermia>

TronicsZone. (2020). PID controller using modern microcontrollers. <https://www.tronicszone.com/blog/pid-controller-using-microcontrollers/>

Wasserman, D. D., Kathard, R., & Healy, M. (2025). Cooling techniques for hyperthermia. *StatPearls*. National Library of Medicine.

Wevolver. (2024). PID explained: Theory, tuning, and implementation of PID controllers.

<https://www.wevolver.com/article/pid-explained-theory-tuning-and-implementation-of-pid-controllers>

Wikipedia. (2026). Hyperthermia. <https://en.wikipedia.org/wiki/Hyperthermia>

Appendix A — Task Breakdown Table

Both team members contributed equally to all aspects of the project. All hardware assembly, firmware development, debugging, and report writing were performed collaboratively throughout the project duration.

Task / Deliverable	Michaël McCorkell	Luka Miljanov
Project planning, requirements definition, and scope statement	✓	✓
Literature review: PID theory, DS18B20 datasheet, PWM fan control	✓	✓
Hardware assembly: breadboard wiring, sensor, fan, LCD, potentiometer		✓
1-Wire bit-bang driver (ds18b20.c/h) development and timing calibration	✓	✓
ADC0 driver (adc.c/h) with hardware + software averaging	✓	✓
Software PWM driver (pwm.c/h), kick-start logic, 10 Hz period selection	✓	✓
Discrete PID controller (pid.c/h) with anti-windup, empirical gain tuning	✓	✓
I2C LCD driver (lcd.c/h) and display formatting	✓	✓
SysTick delay library (delay.c/h) calibration against logic analyzer	✓	✓
System integration: main.c control loop, sensor retry, fault recovery	✓	✓
3D-printed enclosure design (SolidWorks, Figure 2 & 3) and fabrication	✓ (making)	✓ (fabricating)
Testing, debugging, result analysis, and documentation	✓	✓
Final report writing, references, formatting, and submission	✓	✓

Table A1 — Group Collaboration Breakdown. ✓ indicates joint contribution by both authors..

Appendix B — Full Annotated Source Code

The complete firmware source code for the APCS is listed below in full. All files are individually commented and organized. The build environment is Keil MDK-ARM with the CMSIS TM4C123GH6PM device pack.

config.h

```
/*
*****
* config.h - Global configuration and pin definitions
*
* PID Fan Controller - TM4C123GH6PM
*****
*/

#ifndef CONFIG_H
#define CONFIG_H

#include "RTE_Components.h"
#include CMSIS_device_header
#include <stdint.h>

/* ---- LCD ---- */
#define LCD_ADDR      0x28

/* ---- PID Tuning ---- */
#define KP            8.0f
#define KI            0.3f
#define KD            1.0f
```

```

/* ---- Setpoint range from pot (degrees C) ---- */

#define MIN_SET_C      20.0f

#define MAX_SET_C      60.0f


/* ---- Fan limits ---- */

#define MIN_DUTY        10.0f          /* below this the fan stalls; set to match
your fan */

#define MAX_DUTY        100.0f


/* ---- PWM ---- */

/* 16 MHz / 8 = 2 MHz PWM clock.  2 MHz / 20000 = 100 Hz */

#define PWM_LOAD        20000


/* ---- DS18B20 1-Wire commands ---- */

#define OW_CMD_SKIP_ROM    0xCC

#define OW_CMD_CONVERT_T    0x44

#define OW_CMD_READ_SCRATCH 0xBE


/* ---- 1-Wire pin: PE1 ---- */

#define OW_PORT          GPIOE

#define OW_PIN            (1u << 1)

/* Port E bit-banded data register */

#define OW_DATA            (*((volatile uint32_t *) (0x40024000u + (OW_PIN << 2))))

#endif /* CONFIG_H */

```

main.c

```

/*****

```

```

* main.c - PID Fan Controller - TM4C123GH6PM
*
* =====
*
* WIRING
* =====
*
* COMPONENT                | WIRE/PIN      | LAUNCHPAD PIN
* -----|-----|-----
* LCD (I2C addr 0x28)      | SCL           | PB2   (I2C0 SCL)
*                          | SDA           | PB3   (I2C0 SDA)
*                          | VCC           | 5V    (Vss)
*                          | GND           | GND
*                          |               |
* DS18B20 Probe            | Red (VCC)     | 3.3V
*                          | Black (GND)   | GND
*                          | Yellow (DATA) | PE1 + 5k resistor to 3.3V
*                          |               |
* Potentiometer (10k)      | One end       | GND
* (setpoint dial)          | Wiper (mid)   | PE3   (AIN0)
*                          | Other end     | 3.3V
*                          |               |
* 4-Wire PC Fan            | Black         | External 12V PSU GND
*                          | Red/Yellow    | External 12V PSU +12V
*                          | Blue (PWM)    | PB6   (M0PWM0, 25 kHz hardware PWM)
*                          | Green (tach)  | not connected
*                          |               |
* Heartbeat LED            | (onboard)     | PF1   (red LED)
*
* =====

```

```

*   BUILD: Keil uVision with CMSIS TM4C123GH6PM device pack

*   =====

*/

#include "config.h"

#include "delay.h"

#include "lcd.h"

#include "ds18b20.h"

#include "adc.h"

#include "pwm.h"

#include "pid.h"


/* ===== */
/*   GPIO - PF1 red LED heartbeat                               */
/*   ===== */

static void PortF_Init(void)
{
    SYSCCTL->RCGCGPIO |= 0x20u;

    while ((SYSCCTL->PRGPIO & 0x20u) == 0) {}

    GPIOF->DIR |= 0x02u;

    GPIOF->DEN |= 0x02u;
}


/* ===== */
/*   Startup diagnostic: print raw scratchpad bytes on LCD      */
/*   ===== */

static void ShowScratchpadDiag(void)

```

```
{

    uint8_t sp[9];

    char    hex[4];

    int     i;

    OW_Reset();

    OW_WriteByte(OW_CMD_SKIP_ROM);

    OW_WriteByte(OW_CMD_READ_SCRATCH);

    for (i = 0; i < 9; i++)

        sp[i] = OW_ReadByte();

    OW_Reset();

    LCD_Clear();

    LCD_SetCursor(0x00);

    LCD_Print("Raw:");

    for (i = 0; i < 4; i++)

    {

        hex[0] = "0123456789ABCDEF"[sp[i] >> 4];

        hex[1] = "0123456789ABCDEF"[sp[i] & 0x0F];

        hex[2] = ' ';

        hex[3] = '\\0';

        LCD_Print(hex);

    }

    LCD_SetCursor(0x40);

    for (i = 4; i < 9; i++)

    {

        hex[0] = "0123456789ABCDEF"[sp[i] >> 4];

        hex[1] = "0123456789ABCDEF"[sp[i] & 0x0F];

        hex[2] = ' ';
```

```
        hex[3] = '\\0';

        LCD_Print(hex);

    }

    delayMs(4000);
}

/* ===== */
/*  main                                           */
/* ===== */

int main(void)
{
    float    curTempC = 0.0f;
    float    setTempC = 0.0f;
    float    duty      = 0.0f;
    uint32_t adcPot;
    uint8_t  sensorOk = 0;
    uint8_t  retries;

    /* ---- Peripheral init ---- */

    SysTick_Init();

    I2C0_Init_50kHz();

    LCD_Init();

    OW_Init();

    ADC0_Init();

    PWM0_Init();

    PortF_Init();

    /* ---- Splash screen ---- */
```

```
LCD_SetCursor(0x00);

LCD_Print("PID Fan Control");

LCD_SetCursor(0x40);

LCD_Print("  Starting...  ");

delayMs(1500);


/* ---- Probe for DS18B20 with retries ---- */

for (retries = 0; retries < 5; retries++)

{

    if (OW_Reset())

    {

        sensorOk = 1;

        break;

    }

    delayMs(200);

}


if (sensorOk)

{

    LCD_Clear();

    LCD_Print("Sensor found!");

    LCD_SetCursor(0x40);

    LCD_Print("Converting...");

    DS18B20_StartConvert();

    ShowScratchpadDiag();

}

else

{
```

```

    LCD_Clear();

    LCD_Print("NO SENSOR!");

    LCD_SetCursor(0x40);

    LCD_Print("Check PE1 wire");

    delayMs(3000);

}

/* =====

*   Main control loop

*   ===== */

while (1)

{

    /* ---- Temperature acquisition ---- */

    if (sensorOk)

    {

        DS18B20_StartConvert();

        curTempC = DS18B20_ReadTemp();

        if (curTempC < -998.5f)          /* -999: no presence pulse */

        {

            sensorOk = OW_Reset();

            duty      = 0.0f;

        }

        else if (curTempC < -997.5f)    /* -998: power-on default, retry */

        {

            duty = 0.0f;

        }

    }

    else

```

```
{

    /* Try to reacquire sensor */

    sensorOk = OW_Reset();

    if (sensorOk)

    {

        DS18B20_StartConvert();

        curTempC = DS18B20_ReadTemp();

    }

    duty = 0.0f;

}

/* ---- Setpoint from potentiometer ---- */

adcPot    = ADC0_ReadPotAvg();

setTempC = MIN_SET_C + ((float)adcPot / 4095.0f) * (MAX_SET_C - MIN_SET_C);

/* ---- PID ---- */

if (sensorOk && curTempC > -900.0f)

    duty = PID_Compute(setTempC, curTempC, 2.0f); /* ~750ms convert +
~1000ms delay + ~80ms LCD */

else

    duty = 0.0f;

/* ---- Fan output with kick-start ---- */

{

    static uint8_t fanWasOff = 1;

    if (duty > 0.0f && fanWasOff)

    {

        PWM0_KickStart();

        fanWasOff = 0;

    }

}
```

```

    }

    else if (duty <= 0.0f)

    {

        fanWasOff = 1;

    }

    PWM0_SetDuty(duty);

}

/* ---- LCD update ---- */

{

    uint8_t status = sensorOk ? 1u : 0u;

    if (sensorOk && curTempC < -997.5f) status = 2u;

    LCD_Update(curTempC, setTempC, duty, status);

}

/* ---- Heartbeat LED ---- */

GPIOF->DATA ^= 0x02u;

/* ---- Pace the loop to ~2 seconds (hardware PWM runs on its own) ---- */

delayMs(1000);

}

}

```

adc.c

```

/*****

* adc.c - ADC0 Sequencer 3, AIN0 (PE3), 64x hardware averaging

*****/

```

```
#include "config.h"

#include "adc.h"

void ADC0_Init(void)
{
    SYSCTL->RCGCADC |= 0x01;

    SYSCTL->RCGCGPIO |= 0x10;          /* Port E (may already be on) */

    while ((SYSCTL->PRGPIO & 0x10) == 0) {}

    GPIOE->DIR    &= ~0x08;           /* PE3 input */
    GPIOE->AFSEL  |= 0x08;
    GPIOE->DEN    &= ~0x08;
    GPIOE->AMSEL  |= 0x08;

    while ((SYSCTL->PRADC & 0x01) == 0) {}

    ADC0->SAC     = 6;                /* 64x hardware oversampling */

    ADC0->ACTSS   &= ~0x08;           /* disable SS3 during config */
    ADC0->EMUX    &= ~0xF000u;        /* software trigger */
    ADC0->SSMUX3   = 0;               /* AIN0 = PE3 */
    ADC0->SSCTL3   = 0x06;            /* IE0 | END0 */
    ADC0->ACTSS   |= 0x08;            /* re-enable SS3 */
}

uint32_t ADC0_ReadPot(void)
{
    uint32_t result;
```

```

    ADC0->PSSI = 0x08;

    while ((ADC0->RIS & 0x08) == 0) {}

    result = ADC0->SSFIFO3 & 0xFFFFu;

    ADC0->ISC = 0x08;

    return result;
}

/* 8-sample software average on top of hardware averaging */
uint32_t ADC0_ReadPotAvg(void)
{
    uint32_t sum = 0;

    int i;

    for (i = 0; i < 8; i++)
        sum += ADC0_ReadPot();

    return sum / 8u;
}

```

adc.h

```

/*****

* adc.h - ADC0 driver for potentiometer on PE3 (AIN0)

*****/

#ifndef ADC_H
#define ADC_H

#include <stdint.h>

void      ADC0_Init(void);

```

```
uint32_t ADC0_ReadPot(void);

uint32_t ADC0_ReadPotAvg(void);    /* 8-sample software average */

#endif /* ADC_H */
```

ds18b20.c

```
/* ===== */
* ds18b20.c - DS18B20 temperature sensor (1-Wire bit-bang on PE1)
*****/

#include "config.h"
#include "delay.h"
#include "ds18b20.h"

/* ===== */
/* 1-Wire bus initialisation */
/* ===== */

void OW_Init(void)
{
    SYSTCL->RCGCGPIO |= 0x10;    /* enable Port E clock */

    while ((SYSTCL->PRGPIO & 0x10) == 0) {}

    OW_PORT->AFSEL &= ~OW_PIN;
    OW_PORT->AMSEL &= ~OW_PIN;
    OW_PORT->DEN    |=  OW_PIN;
    OW_PORT->PUR    |=  OW_PIN;    /* internal pull-up (backup) */
    OW_PORT->DIR    &= ~OW_PIN;    /* default: input (idle high) */
}
```

```
}

/* ===== */
/* 1-Wire line drive helpers (static - internal use only) */
/* ===== */

static void OW_DriveLow(void)
{
    OW_PORT->DIR |= OW_PIN;

    OW_DATA      = 0;
}

/* Actively drive high during DS18B20 conversion (strong pull-up) */
static void OW_DriveHigh(void)
{
    OW_DATA      = OW_PIN;
    OW_PORT->DIR |= OW_PIN;
}

static void OW_Release(void)
{
    OW_PORT->DIR &= ~OW_PIN;      /* tri-state; external resistor holds high */
}

static uint8_t OW_ReadPin(void)
{
    return (OW_DATA) ? 1u : 0u;
}
```

```
/* ===== */
/* 1-Wire bus primitives */
/* ===== */

uint8_t OW_Reset(void)
{
    uint8_t presence;

    OW_DriveLow();
    delayUs(480);
    OW_Release();
    delayUs(70);
    presence = (OW_ReadPin() == 0) ? 1u : 0u;
    delayUs(410);
    return presence;
}

static void OW_WriteBit(uint8_t bit)
{
    if (bit)
    {
        OW_DriveLow();
        delayUs(6);
        OW_Release();
        delayUs(64);
    }
    else
    {
        OW_DriveLow();
    }
}
```

```
        delayUs(60);

        OW_Release();

        delayUs(10);

    }

}

static uint8_t OW_ReadBit(void)
{
    uint8_t bit;

    OW_DriveLow();

    delayUs(2);

    OW_Release();

    delayUs(12);

    bit = OW_ReadPin();

    delayUs(56);

    return bit;
}

void OW_WriteByte(uint8_t byte)
{
    int i;

    for (i = 0; i < 8; i++)
    {
        OW_WriteBit(byte & 0x01u);

        byte >>= 1;
    }
}
```

```
uint8_t OW_ReadByte(void)
{
    uint8_t byte = 0;
    int i;
    for (i = 0; i < 8; i++)
    {
        byte >>= 1;
        if (OW_ReadBit())
            byte |= 0x80u;
    }
    return byte;
}

/* ===== */
/* DS18B20 high-level API */
/* ===== */

void DS18B20_StartConvert(void)
{
    OW_Reset();

    OW_WriteByte(OW_CMD_SKIP_ROM);

    OW_WriteByte(OW_CMD_CONVERT_T);

    /* Strong pull-up during 12-bit conversion (750 ms max) */
    OW_DriveHigh();
    delayMs(750);
    OW_Release();
}
```

```
float DS18B20_ReadTemp(void)
{
    uint8_t  sp[9];
    int16_t  raw;
    int      i;

    if (!OW_Reset())
        return -999.0f;

    OW_WriteByte(OW_CMD_SKIP_ROM);
    OW_WriteByte(OW_CMD_READ_SCRATCH);

    for (i = 0; i < 9; i++)
        sp[i] = OW_ReadByte();

    OW_Reset();

    /* All-0xFF → no sensor responding */
    if (sp[0] == 0xFF && sp[1] == 0xFF)
        return -999.0f;

    raw = ((int16_t)sp[1] << 8) | sp[0];

    /* 0x0550 = 85.0 °C power-on default → conversion not yet complete */
    if (raw == 0x0550)
        return -998.0f;

    return (float)raw * 0.0625f;
}
```

ds18b20.h

```
/* *****  
 * ds18b20.h - DS18B20 temperature sensor driver (1-Wire bit-bang on PE1)  
 ***** */  
  
#ifndef DS18B20_H  
#define DS18B20_H  
  
#include <stdint.h>  
  
/* ---- 1-Wire bus primitives ---- */  
void    OW_Init(void);  
uint8_t OW_Reset(void);  
void    OW_WriteByte(uint8_t byte);  
uint8_t OW_ReadByte(void);  
  
/* ---- DS18B20 high-level API ---- */  
/*  
 * Triggers a 12-bit temperature conversion.  
 * Actively drives the line high during the 750 ms conversion window  
 * to supply the strong pull-up current required by the sensor.  
 */  
void    DS18B20_StartConvert(void);  
  
/*  
 * Reads the scratchpad and returns temperature in °C.  
 * Return values:
```

```

*   >= -900   : valid temperature

*   -998.0    : power-on default (0x0550) – conversion not yet complete

*   -999.0    : no sensor presence detected

*/

float DS18B20_ReadTemp(void);

#endif /* DS18B20_H */

```

pwm.c

```

/*****

* pwm.c – Hardware PWM on PB6 (M0PWM0, 25 kHz)

*

* PWM Module 0, Generator 0, Output A (M0PWM0) drives PB6.

* The system clock (16 MHz) feeds the PWM counter directly (USEPWMDIV = 0).

*   Period  = 16 000 000 / 25 000 = 640 counts → LOAD = 639

*   Smaller CMPA value → longer HIGH time → higher duty cycle

*

* Port B clock is already on when this init runs (I2C0 init enables it).

*****/

#include "config.h"

#include "delay.h"

#include "pwm.h"

/* ---- PWM0 module registers (raw; avoids relying on CMSIS PWM struct) ---- */

#define SYSCTL_RCGCPWM_R  (*((volatile uint32_t *)0x400FE640u))

#define SYSCTL_RCC_R      (*((volatile uint32_t *)0x400FE060u))

```

```

#define PWM0_ENABLE_R      (*((volatile uint32_t *)0x40028008u))

#define PWM0_0_CTL_R       (*((volatile uint32_t *)0x40028040u))

#define PWM0_0_GENA_R      (*((volatile uint32_t *)0x40028060u))

#define PWM0_0_LOAD_R      (*((volatile uint32_t *)0x40028050u))

#define PWM0_0_CMPA_R      (*((volatile uint32_t *)0x40028058u))


#define SYSCLK_HZ           16000000u

#define PWM_FREQ_HZ        25000u

#define PWM_TOP             ((SYSCLK_HZ / PWM_FREQ_HZ) - 1u)    /* 639 */

/* ===== */

void PWM0_Init(void)
{
    /* 1) Enable PWM0 module clock; clear USEPWMDIV so PWM runs at sysclk */
    SYSCTL_RCGCPWM_R |= (1u << 0);

    SYSCTL_RCC_R      &= ~(1u << 20);    /* USEPWMDIV = 0 */

    delayMs(1);                          /* let clock settle */


    /* 2) PB6 → M0PWM0 alternate function */
    GPIOB->AFSEL |= (1u << 6);

    GPIOB->DEN    |= (1u << 6);

    GPIOB->AMSEL &= ~(1u << 6);

    GPIOB->DIR    |= (1u << 6);

    GPIOB->PCTL   = (GPIOB->PCTL & ~(0xFu << 24)) | (0x4u << 24);


    /* 3) Configure Generator 0 */

    PWM0_0_CTL_R = 0;                    /* disable during setup */

    PWM0_0_GENA_R = 0x00000008Cu;        /* HIGH on LOAD, LOW on CMPA match */

```

```

    PWM0_0_LOAD_R = PWM_TOP;          /* 25 kHz period */

    PWM0_0_CMPA_R = PWM_TOP;          /* start at 0 % (compare == LOAD) */

    /* 4) Start the generator; output stays gated off until duty > 0 */
    PWM0_0_CTL_R |= 0x1u;
}

/* ----- */

void PWM0_SetDuty(float pct)
{
    uint32_t cmp;

    if (pct <= 0.0f)
    {
        PWM0_ENABLE_R &= ~0x1u;      /* gate output off → PB6 stays low */
        return;
    }

    if (pct > 100.0f) pct = 100.0f;

    PWM0_ENABLE_R |= 0x1u;          /* (re-)enable output */

    /* Map duty to compare value.

       At cmp = 0 the pin is HIGH for the full period (100 %).

       At cmp = PWM_TOP the pin goes LOW almost immediately (≈ 0 %). */
    if (pct >= 100.0f)
    {
        cmp = 0u;
    }
}

```

```

    else
    {
        cmp = ((PWM_TOP + 1u) * (uint32_t)(100.0f - pct + 0.5f)) / 100u;
    }

    PWM0_0_CMPA_R = cmp;
}

/* ----- */

void PWM0_KickStart(void)
{
    /* Blast 100 % for 500 ms to overcome motor stiction.

       Caller (main) will call PWM0_SetDuty(duty) immediately after. */

    PWM0_SetDuty(100.0f);

    delayMs(500);
}

```

pwm.h

```

/*****

* pwm.h - Hardware PWM on PB6 (M0PWM0, 25 kHz)

*****/

#ifndef PWM_H
#define PWM_H

void PWM0_Init(void);

void PWM0_SetDuty(float pct); /* 0.0 - 100.0 % ; hardware handles timing */

void PWM0_KickStart(void); /* 500 ms full-power pulse to overcome inertia */

#endif /* PWM_H */

```

pid.c

```

/*****

* pid.c - Discrete PID controller with anti-windup and dead-band clamping
*
* Gains and limits are defined in config.h (KP, KI, KD, MIN_DUTY, MAX_DUTY).
*
* Error convention: error = measured - setpoint
*   → positive error means temperature is ABOVE setpoint → increase fan duty
*****/

#include "config.h"
#include "pid.h"

static float g_integral = 0.0f;
static float g_prevError = 0.0f;

float PID_Compute(float setpoint, float measured, float dt)
{
    float error, pTerm, iTerm, dTerm, output;

    error = measured - setpoint;

    /* Proportional */
    pTerm = KP * error;

    /* Integral with symmetric anti-windup clamp */
    g_integral += error * dt;
    if (g_integral > 80.0f) g_integral = 80.0f;
    if (g_integral < -80.0f) g_integral = -80.0f;

```

```

    iTerm = KI * g_integral;

    /* Derivative */
    dTerm = KD * (error - g_prevError) / dt;
    g_prevError = error;

    output = pTerm + iTerm + dTerm;

    /* Clamp output and clear integral if fan should be off */
    if (output < 0.0f)
    {
        output = 0.0f;
        if (g_integral < 0.0f) g_integral = 0.0f; /* conditional reset */
    }
    else if (output < MIN_DUTY)
    {
        /* Below minimum: either hold at MIN_DUTY (if error warrants) or off */
        output = (error > 1.0f) ? MIN_DUTY : 0.0f;
    }

    if (output > MAX_DUTY) output = MAX_DUTY;

    return output;
}

```

pid.h

```

/*****
 * pid.h - PID controller

```

```

*****/

#ifndef PID_H

#define PID_H

float PID_Compute(float setpoint, float measured, float dt);

#endif /* PID_H */

```

lcd.c

```

/*****

* lcd.c - I2C0 driver (PB2 = SCL, PB3 = SDA) and LCD helper functions
*
* LCD I2C address : LCD_ADDR (0x28) - defined in config.h
* Command prefix  : 0xFE
*****/

#include "config.h"
#include "delay.h"
#include "utils.h"
#include "lcd.h"

/* ===== */
/* I2C0 (PB2 = SCL, PB3 = SDA) */
/* ===== */

void I2C0_Init_50kHz(void)
{
    SYSCTL->RCGCI2C |= 0x01;
    SYSCTL->RCGCGPIO |= 0x02;
    while ((SYSCTL->PRGPIO & 0x02) == 0) {}
}

```

```
    GPIOB->AFSEL |= 0x0C;

    GPIOB->DEN    |= 0x0C;

    GPIOB->ODR    |= 0x08;

    GPIOB->AMSEL  &= ~0x0C;

    GPIOB->PCTL   &= ~0x0000FF00u;

    GPIOB->PCTL   |= 0x00003300u;


    I2C0->MCR    = 0x10;

    I2C0->MTPR   = 15;
}

void I2C0_WriteBuffer(uint8_t slave_addr, uint8_t *data, uint32_t len)
{
    uint32_t i;

    if (len == 0) return;

    I2C0->MSA = (slave_addr << 1) & ~0x01u;

    I2C0->MDR = data[0];

    if (len == 1)
    {
        I2C0->MCS = 0x07;

        while (I2C0->MCS & 0x01) {}

        return;
    }

    I2C0->MCS = 0x03;

    while (I2C0->MCS & 0x01) {}
```

```
    for (i = 1; i < len - 1; i++)
    {
        I2C0->MDR = data[i];

        I2C0->MCS = 0x01;

        while (I2C0->MCS & 0x01) {}

    }

    I2C0->MDR = data[len - 1];

    I2C0->MCS = 0x05;

    while (I2C0->MCS & 0x01) {}

}

/* ===== */
/* LCD low-level helpers */
/* ===== */

void LCD_SendCommand(uint8_t cmd)
{
    uint8_t buf[2] = {0xFE, cmd};

    I2C0_WriteBuffer(LCD_ADDR, buf, 2);

    delayMs(2);
}

void LCD_SendCommand1(uint8_t cmd, uint8_t param)
{
    uint8_t buf[3] = {0xFE, cmd, param};

    I2C0_WriteBuffer(LCD_ADDR, buf, 3);

    delayMs(2);
}
```

```
}

void LCD_Clear(void)
{
    LCD_SendCommand(0x51);
    delayMs(2);
}

void LCD_SetCursor(uint8_t pos)
{
    LCD_SendCommand1(0x45, pos);
}

void LCD_Print(char *str)
{
    while (*str)
    {
        uint8_t ch = (uint8_t)(*str++);
        I2C0_WriteBuffer(LCD_ADDR, &ch, 1);
        delayMs(1);
    }
}

void LCD_ClearLine(uint8_t lineAddr)
{
    int i;
    LCD_SetCursor(lineAddr);
    for (i = 0; i < 16; i++)
    {
```

```

        uint8_t ch = ' ';

        I2C0_WriteBuffer(LCD_ADDR, &ch, 1);

        delayMs(1);

    }
}

/* ===== */
/* LCD initialisation */
/* ===== */

void LCD_Init(void)
{
    delayMs(100);

    LCD_SendCommand(0x41);

    LCD_SendCommand1(0x52, 40);

    LCD_SendCommand1(0x53, 8);

    LCD_Clear();
}

/* ===== */
/* Application-level display update */
/* ===== */
/* status: 0 = no sensor, 1 = normal, 2 = startup / not yet converted */
/* ===== */

void LCD_Update(float curTemp, float setTemp, float duty, uint8_t status)
{
    char buf[8];

    char dutyBuf[5];

```

```
/* ---- Line 1 ---- */

LCD_ClearLine(0x00);

LCD_SetCursor(0x00);

if (status == 0)
{
    LCD_Print("Cur:NO SENSOR");
}

else if (status == 2)
{
    LCD_Print("Cur:STARTUP...");
}

else
{
    LCD_Print("Cur:");

    floatToStr1(curTemp, buf);

    LCD_Print(buf);

    LCD_Print("C ");

    intToStr((uint32_t)(duty + 0.5f), dutyBuf);

    LCD_Print(dutyBuf);

    LCD_Print("%");
}

/* ---- Line 2 ---- */

LCD_ClearLine(0x40);

LCD_SetCursor(0x40);

LCD_Print("Set:");

floatToStr1(setTemp, buf);
```

```

    LCD_Print(buf);

    LCD_Print("C PID");
}

```

lcd.h

```

/*****
 * lcd.h - I2C0 driver (PB2/PB3) and LCD helper API
 *****/

#ifndef LCD_H
#define LCD_H

#include <stdint.h>

/* ---- I2C0 ---- */

void I2C0_Init_50kHz(void);

void I2C0_WriteBuffer(uint8_t slave_addr, uint8_t *data, uint32_t len);

/* ---- LCD ---- */

void LCD_Init(void);

void LCD_Clear(void);

void LCD_ClearLine(uint8_t lineAddr);

void LCD_SetCursor(uint8_t pos);

void LCD_Print(char *str);

void LCD_SendCommand(uint8_t cmd);

void LCD_SendCommand1(uint8_t cmd, uint8_t param);

void LCD_Update(float curTemp, float setTemp, float duty, uint8_t status);

```

```
#endif /* LCD_H */
```

delay.c

```
/* *****  
 * delay.c - SysTick-based microsecond / millisecond delay  
 * ***** */  
  
#include "config.h"  
#include "delay.h"  
  
void SysTick_Init(void)  
{  
    SysTick->CTRL = 0;  
    SysTick->LOAD = 0x00FFFFFF;  
    SysTick->VAL  = 0;  
    SysTick->CTRL = 0x05;          /* enable, system clock, no IRQ */  
}  
  
void delayUs(uint32_t us)  
{  
    uint32_t ticks = us * 16;      /* 16 MHz */  
    if (ticks > 0x00FFFFFF) ticks = 0x00FFFFFF;  
    SysTick->LOAD = ticks - 1;  
    SysTick->VAL  = 0;  
    while ((SysTick->CTRL & 0x10000) == 0) {}  
}
```

```

void delayMs(int n)
{
    int i;

    for (i = 0; i < n; i++)
        delayUs(1000);
}

```

delay.h

```

/*****

* delay.h - SysTick-based microsecond / millisecond delay

*****/

#ifndef DELAY_H
#define DELAY_H

#include <stdint.h>

void    SysTick_Init(void);
void    delayUs(uint32_t us);
void    delayMs(int n);

#endif /* DELAY_H */

```

utils.c

```

/*****

* utils.c - Numeric-to-string helpers (no stdlib / printf dependency)

```

```
*****/

#include "utils.h"

/*
 * floatToStr1 - convert float to string with 1 decimal place.
 * Supports negative values and saturates at ±999.9.
 * buf must be at least 7 bytes.
 */
void floatToStr1(float val, char *buf)
{
    int    intPart, fracPart, i, len;
    char   temp[8];
    uint8_t neg = 0;

    if (val < 0.0f) { neg = 1; val = -val; }
    if (val > 999.9f) val = 999.9f;

    intPart = (int)val;
    fracPart = (int)((val - (float)intPart) * 10.0f + 0.5f);
    if (fracPart >= 10) { intPart++; fracPart = 0; }

    i = 0;
    if (intPart == 0)
    {
        temp[i++] = '0';
    }
    else
    {

```

```

        while (intPart > 0) { temp[i++] = (char)((intPart % 10) + '0'); intPart /=
10; }

    }

    len = 0;

    if (neg) buf[len++] = '-';

    for (i = i - 1; i >= 0; i--)

        buf[len++] = temp[i];

    buf[len++] = '.';

    buf[len++] = (char)(fracPart + '0');

    buf[len] = '\0';
}

/*
 * intToStr - convert unsigned integer to decimal string.
 * buf must be at least 11 bytes.
 */
void intToStr(uint32_t num, char *str)
{
    int i = 0, j;

    char temp[10];

    if (num == 0) { str[0] = '0'; str[1] = '\0'; return; }

    while (num > 0) { temp[i++] = (char)((num % 10) + '0'); num /= 10; }

    for (j = 0; j < i; j++) str[j] = temp[i - j - 1];

    str[i] = '\0';
}

```

utils.h

```
/* *****  
  
 * utils.h - Numeric-to-string helpers (no stdlib required)  
  
***** */  
  
#ifndef UTILS_H  
  
#define UTILS_H  
  
#include <stdint.h>  
  
void floatToStr1(float val, char *buf);    /* 1 decimal place, e.g. "23.5" */  
  
void intToStr(uint32_t num, char *str);  
  
#endif /* UTILS_H */
```